

PictureSelector Android 图片/视频选择器库使用指南

`PictureSelector` 是 Android 端一款功能强大的图片/视频选择器库，支持拍照、相册选择、裁剪、压缩、多图选择、视频截取等核心功能，适配性好且定制化程度高，是开发中替代原生相册选择的常用方案。下面从**核心使用步骤**、**关键配置**、**结果回调**三个维度做简单讲解：

一、基础集成（Gradle）

首先在项目中引入依赖（以最新版为例，可去 Github 确认版本号）：

Code block

```
1  // 根目录 build.gradle
2  allprojects {
3      repositories {
4          maven { url 'https://jitpack.io' }
5      }
6  }
7
8  // 模块 build.gradle
9  dependencies {
10     // 核心库
11     implementation
12     'com.github.LuckSiege.PictureSelector:picture_library:v3.11.1'
13     // 如需裁剪/压缩功能，可选引入（部分版本已内置）
14     implementation
15     'com.github.LuckSiege.PictureSelector:picture_compress:v3.11.1'
16     implementation 'com.github.LuckSiege.PictureSelector:picture_crop:v3.11.1'
17 }
```

二、核心使用步骤

1. 权限申请（必做）

`PictureSelector` 依赖存储、相机权限，需先申请（Android 6.0+ 动态权限）：

- 权限清单（`AndroidManifest.xml`）：

Code block

```
1  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

```
2 <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
3 <uses-permission android:name="android.permission.CAMERA" />
```

- 动态申请（推荐用 `EasyPermissions` 或原生 API）：

Code block

```
1 // 原生动态权限示例 (Activity 中)
2 if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) !=
    PackageManager.PERMISSION_GRANTED
3     || ContextCompat.checkSelfPermission(this,
    Manifest.permission.READ_EXTERNAL_STORAGE) !=
    PackageManager.PERMISSION_GRANTED) {
4     ActivityCompat.requestPermissions(this, new String[]
        {Manifest.permission.CAMERA, Manifest.permission.READ_EXTERNAL_STORAGE},
        100);
5 }
```

2. 调用选择器（核心代码）

分为**相册选择**和**拍照**两种场景，通过 `PictureSelector.create()` 构建配置：

场景1：相册选择图片/视频

Code block

```
1 // Activity 中调用
2 PictureSelector.create(this)
3     .openGallery(PictureMimeType.ofImage()) // 选择图片 (ofVideo() 选视频,
    ofAll() 图片+视频)
4     .maxSelectNum(9) // 最大选择数 (默认9)
5     .minSelectNum(1) // 最小选择数 (默认1)
6     .imageSpanCount(4) // 相册网格列数
7     .isCamera(false) // 是否显示拍照按钮
8     .enableCrop(true) // 是否开启裁剪
9     .cropAspectRatio(1,1) // 裁剪比例 (1:1、4:3等)
10    .compress(true) // 是否压缩
11    .forResult(101); // 请求码, 用于回调接收结果
```

场景2：直接拍照

Code block

```
1 PictureSelector.create(this)
2     .openCamera(PictureMimeType.ofImage()) // 拍照 (ofVideo() 拍视频)
3     .enableCrop(true) // 拍完是否裁剪
```

```
4         .compress(true) // 是否压缩
5         .forResult(102); // 请求码
```

3. 接收选择结果（回调处理）

在 `onActivityResult`（旧版）或 `registerForActivityResult`（新版）中接收结果：

方式1：旧版（Activity 重写 `onActivityResult`）

Code block

```
1  @Override
2  protected void onActivityResult(int requestCode, int resultCode, @Nullable
    Intent data) {
3      super.onActivityResult(requestCode, resultCode, data);
4      if (resultCode == RESULT_OK) {
5          // 根据请求码获取选择结果
6          if (requestCode == 101 || requestCode == 102) {
7              List<LocalMedia> selectList =
    PictureSelector.obtainMultipleResult(data);
8              // LocalMedia 是封装的媒体信息类，包含路径、宽高、是否裁剪等
9              for (LocalMedia media : selectList) {
10                 String path = media.getRealPath(); // 获取真实路径（推荐）
11                 // 后续：加载图片（Glide/Picasso）、上传等操作
12                 Glide.with(this).load(path).into(imageView);
13             }
14         }
15     }
16 }
```

方式2：新版（Activity Result API，推荐）

Code block

```
1  // 1. 定义回调
2  ActivityResultLauncher<Intent> launcher = registerForActivityResult(
3      new ActivityResultContracts.StartActivityForResult(),
4      result -> {
5          if (result.getResultCode() == RESULT_OK && result.getData() !=
    null) {
6              List<LocalMedia> selectList =
    PictureSelector.obtainMultipleResult(result.getData());
7              // 处理结果...
8          }
9      });
10
11 // 2. 调用选择器时替换 forResult
```

```
12  PictureSelector.create(this)
13      .openGallery(PictureMimeType.ofImage())
14      .maxSelectNum(1)
15      .forResult(launcher); // 传入 launcher
```

三、关键配置说明（常用）

配置方法	作用	示例值
<code>openGallery()</code>	打开相册	<code>PictureMimeType.ofImage()</code>
<code>openCamera()</code>	打开相机	<code>PictureMimeType.ofVideo()</code>
<code>maxSelectNum()</code>	最大选择数量	9（默认）
<code>isSingleMode()</code>	是否单选	true/false
<code>enableCrop()</code>	是否开启裁剪	true
<code>cropAspectRatio()</code>	裁剪比例	1,1（正方形）
<code>compress()</code>	是否压缩	true
<code>isCamera()</code>	相册页是否显示拍照按钮	true（默认）
<code>setImageEngine()</code>	自定义图片加载引擎	<code>new GlideEngine()</code>

四、注意事项

- 1. 路径问题: `LocalMedia` 提供了 `getPath()`（旧路径）和 `getRealPath()`（真实路径），推荐用 `getRealPath()` 避免适配问题；
- 2. Android 10+ 适配: 需在 `AndroidManifest.xml` 的 `<application>` 中添加 `android:requestLegacyExternalStorage="true"`（或适配作用域存储）；
- 3. 自定义样式: 可通过 `theme()` 方法自定义选择器的主题（颜色、文字等）；
- 4. 混淆配置: 如需混淆，需在 `proguard-rules.pro` 中添加：

```
Code block
1  -keep class com.luck.picture.lib.entity.LocalMedia { *; }
```

五、核心优势

- 支持多图/视频选择、混合选择；

- 内置裁剪、压缩、旋转等功能，无需额外集成；
- 适配 Android 各版本（包括 13+ 权限）；
- 可自定义 UI、选择规则、图片加载引擎（如替换为 Glide/Coil）。

总结

`PictureSelector` 的核心使用流程：**集成依赖** → **申请权限** → **配置选择器参数** → **调用选择器** → **接收并处理结果**。日常开发中，只需根据需求调整配置（如是否裁剪、单选/多选），即可快速实现图片/视频选择功能。

如需更复杂的定制（如自定义相册样式、视频截取），可参考其 [官方文档](#)。

（注：文档部分内容可能由 AI 生成）